

Algoritmos e Estruturas de Ordenação

Engenharia Informática · 2º Ano · 2026

```
{ sort(); }
```

Agenda

Conteúdos apresentados neste trabalho

01

Conceitos Fundamentais

Algoritmo, estrutura de dados, ordenação, chave, estabilidade

02

Classificação dos Algoritmos

Ordenação interna/externa · Complexidade computacional

03

Principais Algoritmos

Bubble, Selection, Insertion, Shell, Merge e Quick Sort

04

Metodologia Experimental

Ambiente de testes · Métricas de avaliação

05

Aplicações Práticas

Bases de dados, motores de busca, banca, saúde, e-commerce

01 Conceitos Fundamentais

Algoritmo

Sequência finita de instruções para resolver um problema. Avaliado pelo tempo de execução e memória utilizada.

Estrutura de Dados

Forma de organizar dados para facilitar inserção, remoção, pesquisa e ordenação com eficiência.

Ordenação

Organizar dados por critério definido (crescente, alfabético, cronológico) para melhorar acesso e processamento.

Chave de Ordenação

Atributo que define a posição dos elementos.
Ex: nome, matrícula, média, data de nascimento.

Estabilidade

Preserva a ordem relativa de elementos com chaves iguais. Insertion, Merge e Bubble Sort são estáveis.

02 Classificação dos Algoritmos

Quanto à Localização

Ordenação Interna

Ordenação Externa

Quanto à Complexidade

Simples / Elementares

$O(n^2)$

Bubble, Selection, Insertion Sort

Eficientes / Avançados

$O(n \log n)$

Shell, Merge, Quick, Heap Sort

Lineares

$O(n)$

Counting, Radix, Bucket Sort

03 Algoritmos Elementares

Bubble Sort

Compara elementos adjacentes e realiza trocas até ordenar toda a lista.

Melhor caso: $O(n)$

Caso médio: $O(n^2)$

Pior caso: $O(n^2)$

Estável

Selection Sort

Seleciona o menor elemento da parte não ordenada e posiciona-o corretamente.

Melhor caso: $O(n^2)$

Caso médio: $O(n^2)$

Pior caso: $O(n^2)$

Instável

Insertion Sort

Insere cada elemento na posição correta da parte já ordenada. Ideal para listas quase ordenadas.

Melhor caso: $O(n)$

Caso médio: $O(n^2)$

Pior caso: $O(n^2)$

Estável

03 Algoritmos Avançados

Shell Sort

Evolução do Insertion Sort

Compara elementos distantes entre si usando intervalos (gaps), reduzindo movimentações.

Melhor: $O(n \log n)$

Médio: **Variável**

Pior: $O(n^2)$

Instável

Merge Sort

Divisão e Conquista

Divide o vetor em partes menores, ordena-as individualmente e combina de forma ordenada.

Melhor: $O(n \log n)$

Médio: $O(n \log n)$

Pior: $O(n \log n)$

Estável

Quick Sort

Divisão e Conquista

Escolhe um pivô e reorganiza os elementos: menores à esquerda e maiores à direita, recursivamente.

Melhor: $O(n \log n)$

Médio: $O(n \log n)$

Pior: $O(n^2)$

Instável

Comparação dos Algoritmos

Algoritmo	Melhor	Médio	Pior	Estável	Memória
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	✓	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	✗	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	✓	$O(1)$
Shell Sort	$O(n \log n)$	Variável	$O(n^2)$	✗	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	✓	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	✗	$O(\log n)$

04 Metodologia Experimental

Ambiente de Testes

Cenário 1

Lista ordenada em ordem crescente

Cenário 2

Lista ordenada em ordem decrescente

Cenário 3

Lista com valores aleatórios

Métricas de Avaliação



Tempo de execução



Número de comparações



Número de trocas (movimentações)



Consumo de memória

05 Aplicações Práticas



Bases de Dados

Organização por critérios; otimização de índices e consultas.



Motores de Busca

Classificação de resultados por relevância e qualidade.



Sistemas Hospitalares

Triagem de pacientes por prioridade médica e gravidade.



Sistemas Bancários

Organização de transações, extratos e relatórios financeiros.



Comércio Eletrónico

Ordenação por preço, popularidade e avaliação de clientes.



Redes Sociais

Seleção de conteúdo por relevância, interações e preferências.

Código: Bubble Sort

```
1  import time
2
3  LISTA = [19, 7, 20, 6, 15, 10, 16, 13, 11, 8, 6, 5]
4
5  def bubble_sort(arr):
6      comp = trocas = 0
7      n = len(arr)
8      print(f"Entrada: {arr}")
9
10     for i in range(n):
11         for j in range(0, n-i-1):
12             comp += 1
13             if arr[j] > arr[j+1]:
14                 arr[j], arr[j+1] = arr[j+1], arr[j]
15                 trocas += 1
16                 print(f"Troca {trocas}: {arr}")
17     return arr, comp, trocas
18
19 if __name__ == "__main__":
20     inicio = time.perf_counter()
21     arr = LISTA.copy()
22     resultado, comp, trocas = bubble_sort(arr)
23     tempo = (time.perf_counter() - inicio) * 1000
24
25     print(f"\n**Bubble Sort**")
26     print(f"Saída: {resultado}")
27     print(f"Comparações: {comp} | Trocas: {trocas} | Tempo: {tempo:.3f}ms")
```

Código: Selection Sort

```
1 import time
2
3 LISTA = [19, 7, 20, 6, 15, 10, 16, 13, 11, 8, 6, 5]
4
5 def selection_sort(arr):
6     comp = trocas = 0
7     print(f"Entrada: {arr}")
8
9     for i in range(len(arr)):
10         min_idx = i
11         for j in range(i+1, len(arr)):
12             comp += 1
13             if arr[j] < arr[min_idx]:
14                 min_idx = j
15         if min_idx != i:
16             arr[i], arr[min_idx] = arr[min_idx], arr[i]
17             trocas += 1
18             print(f"Troca {trocas}: {arr}")
19     return arr, comp, trocas
20
21 if __name__ == "__main__":
22     inicio = time.perf_counter()
23     arr = LISTA.copy()
24     resultado, comp, trocas = selection_sort(arr)
25     tempo = (time.perf_counter() - inicio) * 1000
26
27     print(f"\n**Selection Sort**")
28     print(f"Saída: {resultado}")
29     print(f"Comparações: {comp} | Trocas: {trocas} | Tempo: {tempo:.3f}ms")
30
```

Código: Insertion Sort

```
1  import time
2
3  LISTA = [19, 7, 20, 6, 15, 10, 16, 13, 11, 8, 6, 5]
4
5  def insertion_sort(arr):
6      comp = trocas = 0
7      print(f"Entrada: {arr}")
8
9      for i in range(1, len(arr)):
10         key = arr[i]
11         j = i-1
12         while j >= 0 and arr[j] > key:
13             comp += 1
14             arr[j+1] = arr[j]
15             trocas += 1
16             j -= 1
17         if j >= 0: comp += 1
18         arr[j+1] = key
19         print(f"Inserir {key}: {arr}")
20     return arr, comp, trocas
21
22 if __name__ == "__main__":
23     inicio = time.perf_counter()
24     arr = LISTA.copy()
25     resultado, comp, trocas = insertion_sort(arr)
26     tempo = (time.perf_counter() - inicio) * 1000
27
28     print(f"\n**Insertion Sort**")
29     print(f"Saída: {resultado}")
30     print(f"Comparações: {comp} | Moves: {trocas} | Tempo: {tempo:.3f}ms")
31
```

Código: Shell Sort

```
1  import time
2
3  LISTA = [19, 7, 20, 6, 15, 10, 16, 13, 11, 8, 6, 5]
4
5  def shell_sort(arr):
6      comp = trocas = 0
7      n = len(arr)
8      print(f"Entrada: {arr}")
9
10     gap = n // 2
11     while gap > 0:
12         print(f"\ngap = {gap}")
13         for i in range(gap, n):
14             temp = arr[i]
15             j = i
16             while j >= gap and arr[j - gap] > temp:
17                 comp += 1
18                 arr[j] = arr[j - gap]
19                 trocas += 1
20                 j -= gap
21             if j >= gap: comp += 1
22             arr[j] = temp
23             print(f"Insere {temp}: {arr}")
24         gap //= 2
25
26     return arr, comp, trocas
27
28 if __name__ == "__main__":
29     inicio = time.perf_counter()
30     arr = LISTA.copy()
31     resultado, comp, trocas = shell_sort(arr)
32     tempo = (time.perf_counter() - inicio) * 1000
33
34     print(f"\n**Shell Sort**")
35     print(f"Saída: {resultado}")
36     print(f"Comparações: {comp} | Moves: {trocas} | Tempo: {tempo:.3f}ms")
```

Código: Merge Sort

```
1 import random
2 import time
3
4 #LISTA = [random.randint(1, 20) for _ in range(10)]
5 #LISTA = [random.randint(1, 20) for _ in range(100)]
6 #LISTA = [random.randint(1, 20) for _ in range(1000)]
7
8 LISTA = [19, 7, 20, 6, 15, 10, 16, 13, 11, 8, 6, 5]
9
10 def merge_sort(arr):
11     comp = trocas = [0, 0]
12     print(f"Entrada: {arr}")
13
14     def _merge(a):
15         if len(a) > 1:
16             mid = len(a)//2
17             L, R = a[:mid], a[mid:]
18             _merge(L); _merge(R)
19             i=j=k=0
20             while i < len(L) and j < len(R):
21                 comp[0] += 1
22                 if L[i] <= R[j]:
23                     a[k] = L[i]; i += 1
24                 else:
25                     a[k] = R[j]; j += 1
26                 k += 1; trocas[0] += 1
27             while i < len(L):
28                 a[k] = L[i]; i+=1; k+=1; trocas[0]+=1
29             while j < len(R):
30                 a[k] = R[j]; j+=1; k+=1; trocas[0]+=1
31             print(f"Merge: {a}")
32         return a
33
34     _merge(arr)
35     return arr, comp[0], trocas[0]
36
37 if __name__ == "__main__":
38     inicio = time.perf_counter()
39     arr = LISTA.copy()
40     resultado, comp, trocas = merge_sort(arr)
41     tempo = (time.perf_counter() - inicio) * 1000
42
43     print(f"\n**Merge Sort**")
44     print(f"\nLista original: {LISTA}")
45     print(f"\nSaída: {resultado}")
46     print(f"\nComparações: {comp} | Movimentos: {trocas} | Tempo: {tempo:.3f}ms")
```

Código: Quick Sort

```
1  import time
2
3  LISTA = [19, 7, 20, 6, 15, 10, 16, 13, 11, 8, 6, 5]
4
5  def quick_sort(arr):
6      comp = trocas = [0, 0]
7      print(f"Entrada: {arr}")
8
9      def _quick(a):
10         if len(a) <= 1: return a
11         pivot = a[len(a)//2]
12         left, mid, right = [], []
13         for x in a:
14             comp[0] += 1
15             if x < pivot: left.append(x)
16             elif x == pivot: mid.append(x)
17             else: right.append(x)
18         trocas[0] += len(a)
19         res = _quick(left) + mid + _quick(right)
20         print(f"Partição pivô {pivot}: {res}")
21         return res
22
23     return _quick(arr), comp[0], trocas[0]
24
25 if __name__ == "__main__":
26     inicio = time.perf_counter()
27     arr = LISTA.copy()
28     resultado, comp, trocas = quick_sort(arr)
29     tempo = (time.perf_counter() - inicio) * 1000
30
31     print(f"\n**Quick Sort**")
32     print(f"Saída: {resultado}")
33     print(f"Comparações: {comp} | Moves: {trocas} | Tempo: {tempo:.3f}ms")
```

Conclusões

A ordenação é essencial para a eficiência dos sistemas computacionais modernos.

Cada algoritmo tem características próprias — não há uma solução universal.

Algoritmos simples ($O(n^2)$) são adequados para fins didáticos e listas pequenas.

Algoritmos avançados ($O(n \log n)$) são preferíveis para grandes volumes de dados.

Os algoritmos de ordenação estão presentes em praticamente todas as áreas tecnológicas.